PROJECT DESCRIPTION

In this lab activity, I performed tasks associated with using tcpdump to capture network traffic. I captured the data in a packet capture (p-cap) file and then examined the contents of the captured packet data to focus on specific types of traffic.

SCENARIO

I am a network analyst who needs to use `tcpdump` to capture and analyze live network traffic from a Linux virtual machine.

FIRST STEP: IDENTIFY NETWORK INTERFACES

In this task, I identified network interfaces that can be used to capture network packet data.

- I used **`ifconfig`** to identify the interfaces that are available, this command returns the following output :

```
analyst@943bf9bca394:~$ sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1460
        inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255
.255
        ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
        RX packets 842  bytes 14002103 (13.3 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 488  bytes 43045 (42.0 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 110  bytes 13748 (13.4 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 110  bytes 13748 (13.4 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

- I used **`tcpdump`** to identify the interface options available for packet capture, this command also allows me to identify which network interfaces are available. This may be useful on systems that do not include the **`ifconfig`** command:

```
analyst@943bf9bca394:~$ sudo tcpdump -D
1.eth0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running
]
3.lo [Up, Running, Loopback]
4.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
5.nflog (Linux netfilter log (NFLOG) interface) [none]
6.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
7.dbus-system (D-Bus system bus) [none]
8.dbus-session (D-Bus session bus) [none]
```

SECOND STEP: INSPECT THE NETWORK TRAFFIC OF A NETWORK INTERFACE WITH tcpdump

In this task, I used `tcpdump` to filter live network packet traffic on an interface.

- Filter live network packet data from the `eth0` interface with `tcpdump`: **sudo tcpdump -i eth0 -v -c5.** This command will run `tcpdump` with the following options:
- **`-i eth0`:** Capture data specifically from the `eth0` interface.
- **`-v`:** Display detailed packet data.
- **`-c5`:** Capture 5 packets of data.

```
analyst@943bf9bca394:~$ sudo tcpdump -i eth0 -v -c5
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot
length 262144 bytes
08:29:00.011837 IP (tos 0x0, ttl 64, id 2931, offset 0, flags [DF]
, proto TCP (6), length 121)
    943bf9bca394.5000 > nginx-us-west1-c.c.qwiklabs-terminal-vms-p
rod-00.internal.54698: Flags [P.], cksum 0x591e (incorrect -> 0xbf
74), seq 3689903553:3689903622, ack 1425808922, win 998, options [
nop,nop,TS val 167884883 ecr 3400867066], length 69
08:29:00.013090 IP (tos 0x0, ttl 63, id 18055, offset 0, flags [DF
], proto TCP (6), length 52)
    nginx-us-west1-c.c.qwiklabs-terminal-vms-prod-00.internal.5469
8 > 943bf9bca394.5000: Flags [.], cksum 0xc38d (correct), ack 69,
win 507, options [nop,nop,TS val 3400867128 ecr 167884883], length
 0
08:29:00.080161 IP (tos 0x0, ttl 64, id 3605, offset 0, flags [DF]
, proto UDP (17), length 71)
    943bf9bca394.42660 > metadata.google.internal.domain: 61915+ P
TR? 137.0.22.172.in-addr.arpa. (43)
08:29:00.084346 IP (tos 0x0, ttl 64, id 2932, offset 0, flags [DF]
, proto TCP (6), length 142)
    943bf9bca394.5000 > nginx-us-west1-c.c.qwiklabs-terminal-vms-p
rod-00.internal.54698: Flags [P.], cksum 0x5933 (incorrect -> 0xb0
8a), seq 69:159, ack 1, win 998, options [nop,nop,TS val 167884955
 ecr 3400867128], length 90
```

THIRD STEP: CAPTURE NETWORK TRAFFIC WITH tcpdump

In this task, I used **`tcpdump`** to save the captured network data to a packet capture file.

In the previous command, I used **`tcpdump`** to stream all network traffic. Here, I used a filter and other `tcpdump` configuration options to save a small sample that contains only web (TCP port 80) network packet data.

1. I Captured packet data into a file called **`capture.pcap`**:**sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap &**

This command will run `tcpdump` in the background with the following options:

- **`-i eth0`:** Capture data from the `eth0` interface.
- **`-nn`:** Do not attempt to resolve IP addresses or ports to names.
- **`-c9`:** Capture 9 packets of data and then exit.
- **`port 80`:** Filter only port 80 traffic. This is the default HTTP port.
- **`-w capture.pcap`:** Save the captured data to the named file.
- **`&`:** This is an instruction to the Bash shell to run the command in the background.

```
analyst@943bf9bca394:~$ sudo tcpdump -i eth0 -nn -c9 port 80 -w ca
pture.pcap &
[1] 13563
analyst@943bf9bca394:~$ tcpdump: listening on eth0, link-type EN10
MB (Ethernet), snapshot length 262144 bytes
```

2. I Used **`curl`** to generate some HTTP (port 80) traffic & verified the packet data capture.

```
analyst@943bf9bca394:~$ curl opensource.google.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html;cha
rset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="https://opensource.google/">here</A>.
</BODY></HTML>
analyst@943bf9bca394:~$ 9 packets captured
10 packets received by filter
0 packets dropped by kernel
```

```
analyst@943bf9bca394:~$ ls -l capture.pcap
-rw-r--r-- 1 tcpdump tcpdump 1401 Dec 21 08:45 capture.pcap
```

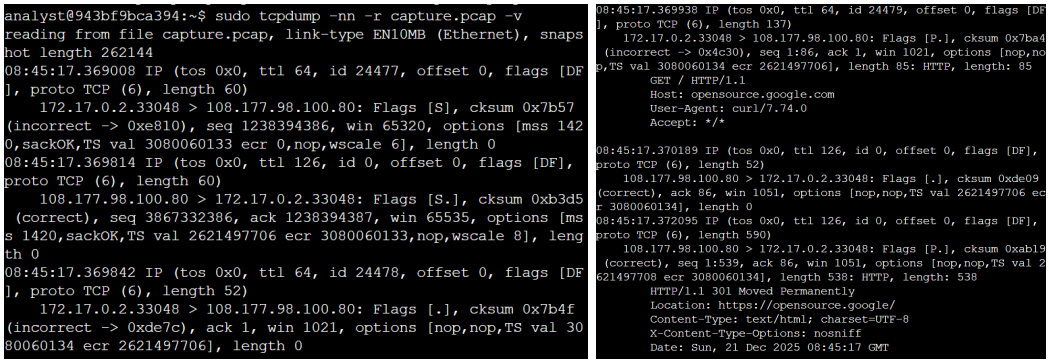
FINAL STEP: FILTER THE CAPTURED PACKET DATA

In this task, I used **`tcpdump`** to filter data from the packet capture file I saved previously.

1. I Used the **`tcpdump`** command to filter the packet header data from the **`capture.pcap`** capture file: **sudo tcpdump -nn -r capture.pcap -v**

This command will run **`tcpdump`** with the following options:

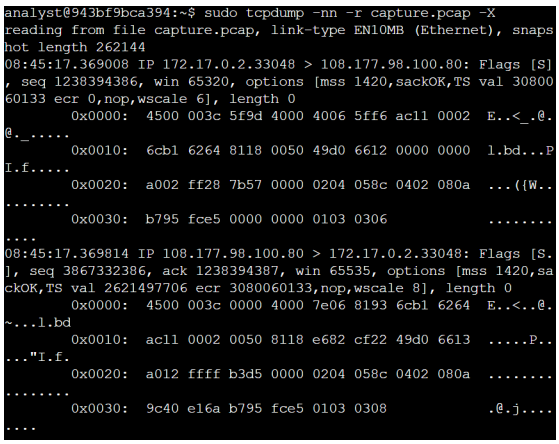
- **`-nn`**: Disable port and protocol name lookup.
- **`-r`**: Read capture data from the named file.
- **`-v`**: Display detailed packet data.

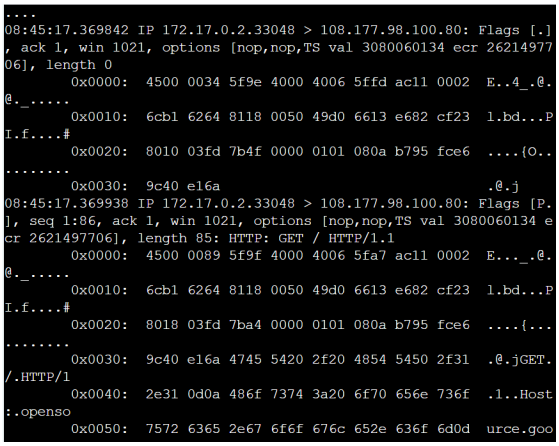


I must specify the **`-nn`** switch again here, as I want to make sure **`tcpdump`** does not perform name lookups of either IP addresses or ports, since this can alert threat actors.

2. I Used the **`tcpdump`** command to filter the extended packet data from the **`capture.pcap`** capture file: **sudo tcpdump -nn -r capture.pcap -X.** This command will run `tcpdump` with the following options:

- **`-nn`:** Disable port and protocol name lookup.
- **`-r`:** Read capture data from the named file.
- **`-X`:** Display the hexadecimal and ASCII output format packet data.





CONCLUSION

I have gained practical experience to enable you to

- identify network interfaces,
- use the `tcpdump` command to capture network data for inspection,
- interpret the information that `tcpdump` outputs regarding a packet, and
- save and load packet data for later analysis.